SLOTBOOK

Reliable appointment booking by design

Full-stack TypeScript / React / Fastify / PostgreSQL

ORIGINAL ASSIGNMENT ALIGNMENT

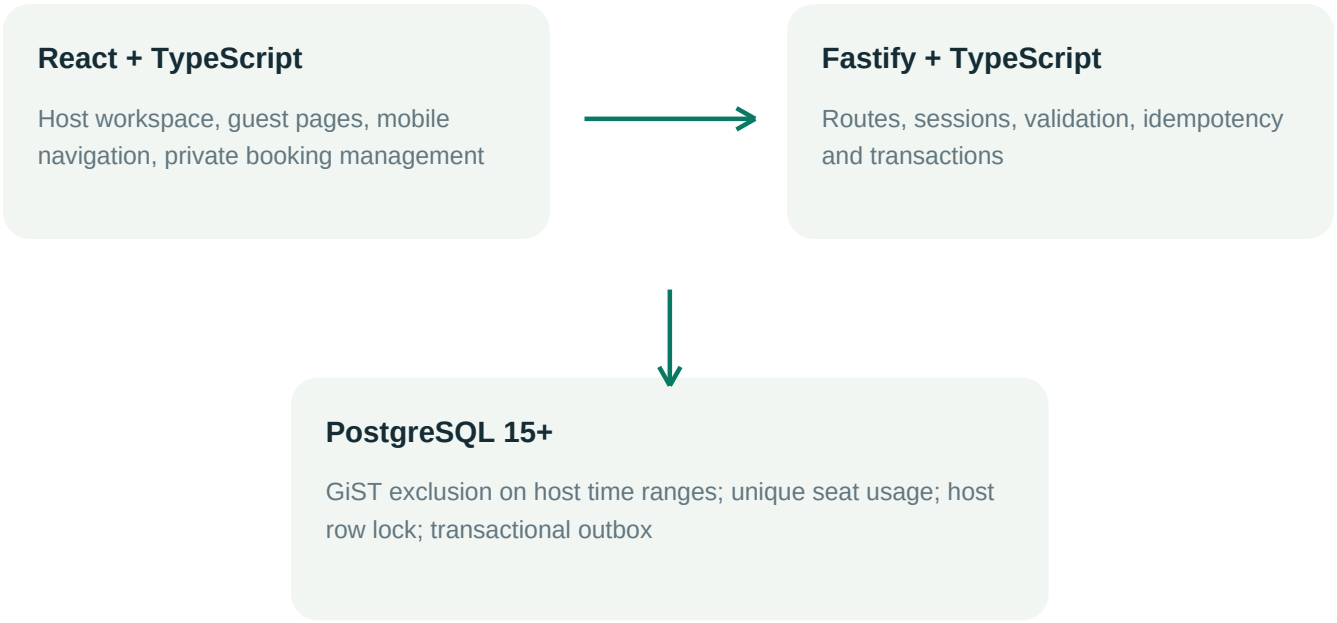
- Host dashboard, event types and weekly/date availability
- Public one-on-one and group appointment booking
- Five-minute holds, confirmation, cancellation and rescheduling
- PostgreSQL overlap/seat constraints, idempotency and server-clock expiry
- Documented tests, notification outbox and AI usage

Verification note: a prior Windows build and PostgreSQL startup succeeded, but the coordinated release 2 changes have not yet undergone a full new Windows/Postgres regression run. No unverified test number is reported as a pass.

Public GitHub URL: to be inserted after repository publication

Architecture and request path

One PostgreSQL source of truth; application components coordinate users but never override the booking constraints.



Request path

Guest chooses a slot in the host schedule. The API validates it against minimum notice and horizon, obtains a host-row transaction lock and persists a held booking. PostgreSQL enforces non-overlapping active sessions for a host and one active booking per seat.

Separation of responsibilities

React owns display and recovery UX. Fastify owns authentication and command orchestration. PostgreSQL owns durable booking integrity, expiration comparisons and idempotent results. The application does not use Redis locks.

Data model and invariants

Tables are linked by ownership and booking identity; the database, not the UI, rejects conflicting writes.

Table	Purpose / critical columns
hosts, auth_sessions	UUID host identity, IANA zone; hashed passwords and HttpOnly session token hashes
event_types	Host-owned duration (15-120), capacity, buffers, notice, horizon and enabled state
weekly_availability	Multiple validated non-overlapping windows per weekday
date_overrides	Host-local date, day-off flag and optional custom windows
booking_sessions	Host/event association; UTC start/end, range including buffers; session capacity
session_seats	Seat numbers 1 to capacity; guard forbids exceeding capacity
bookings	Seat reference, held/confirmed/expired/cancelled status, expiry, token hash, version
idempotency_records	Unique operation scope/key, request fingerprint, committed status and JSON result
notification_outbox	Durable confirmation, reschedule and cancellation event payloads

PostgreSQL safeguards

GiST exclusion rejects overlapping active host sessions. A unique partial index allows only one active held/confirmed booking per seat. Version checks make stale reschedules fail. Release/active booking consistency triggers are deferred to transaction commit.

Release 2 migration

A forward-only migration adds a composite (host_id, event_type_id) foreign key, immutable session schedule/capacity fields and expiry indexes. Existing production tables and bookings are not reset.

Booking lifecycle and atomic actions

01 / AVAILABLE

Host publishes working hours, event settings and optional date overrides

02 / HELD

A guest selects a slot; one seat is reserved for five minutes in a single transaction

03 / CONFIRMED

The guest submits name/email; PostgreSQL clock validates hold expiry

04 / RESCHEDULE

A transaction switches the same booking to a free seat and releases the old session if empty

05 / CANCEL / EXPIRE

Cancellation immediately frees a seat; expired holds are excluded from capacity and later cleaned

Atomic rollback

If a reschedule target is taken, the write fails and its complete transaction rolls back. The guest keeps the original confirmed appointment, rather than losing the reservation first.

Private access

Hosts use authenticated HttpOnly sessions; guests receive a private management token. Passwords and booking token hashes are never returned through the host directory.

Eight engineering decisions

Trade-offs focus on consistency, recoverability and a limited assignment timebox.

Decision	Reason / alternative rejected
1. Native PostgreSQL constraints	Reject overlaps and duplicate active seats even if API validation misses a race.
2. Host-row lock for writes	Serialize capacity decisions; simpler than distributed locking or optimistic retries.
3. Idempotency persisted with command	If HTTP response is lost, same key replays committed outcome instead of double-creating.
4. Database-clock expiry	Server/device clocks are display hints; expiry validity is checked at the database.
5. UTC instants / host-local schedule	Keep events absolute while supporting geographically distant guests.
6. Immutable session promises	Direct SQL cannot silently rewrite a booked time or capacity.
7. Transactional notification outbox	Events exist with committed booking changes even if mail delivery is absent.
8. Safe local seeded auth	Original brief allows seeded host login; avoids unrelated account onboarding scope.

Frontend recoverability

The browser preserves retry identity and pending guest operations within the same tab; it offers explicit hold release and retry. A previous incomplete UI created a new key on each attempt, risking confusing responses after a successful-but-lost request.

Failure modes and verification

These tests are authored; live PostgreSQL results for release 2 remain pending.

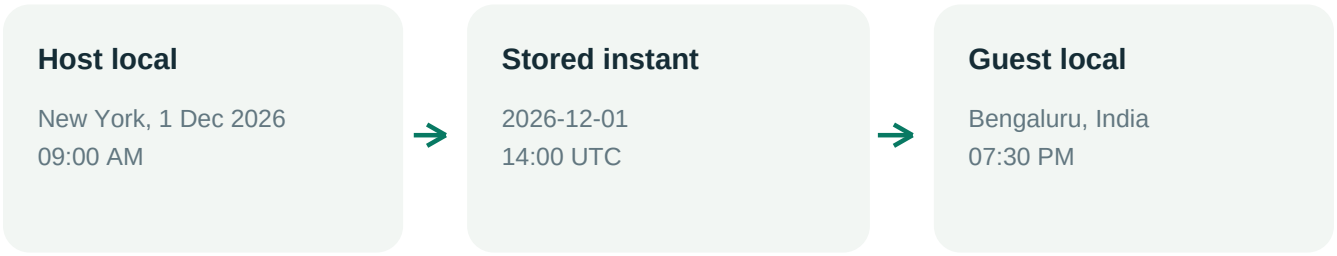
Failure	Invariant / technique	Proof / expected assertion
Two customers / same slot	Exclusion/host lock	50 competing HTTP holds; 1 success, 49 conflicts expected
Four-seat group / five guests	Seat index + capacity	Four holds permitted; fifth rejected
Lost response after commit	Idempotency + retained key	Retry returns same booking identity/hold token
Guest abandons page	Five-minute TTL + cleanup	Expired holds excluded from capacity
Reschedule to occupied time	Atomic rollback + version	Original booking remains unchanged
User/browser clock differs	Database clock	Invalid hold confirmation rejected
Invalid DST local time	Strict IANA conversion	Spring gap and autumn fold skipped
Cross-host event mismatch	Composite foreign key	Direct SQL violation rejected
Host logs out / expired session	Hash-based session revoke	Host routes return unauthorized

Test philosophy

A schema migration succeeding is not proof of contention safety. The integration suite exercises real PostgreSQL writes, including direct SQL bypassing Fastify; a separate HTTP stress script measures 50 competing attempts and counts persisted active bookings.

Time-zone correctness with DST

Schedule generation is based on the host IANA time zone, while guest views may use another zone.



Generate in host-local working hours

For a given host-local day, start with the weekly windows and date override. Convert each allowed local start/end into an absolute UTC interval, then apply event duration, buffers, notice, horizon and occupied capacity.

Display in selected guest time zone

The API returns an ISO UTC start, host-local label, guest-local label and available seats. Selecting a different time zone changes the label, not the instant stored in PostgreSQL.

DST gap - America/New_York (8 March 2026)

At the spring transition, 02:30 does not exist. The strict local-to-UTC converter returns no candidate; that slot is skipped rather than shifted unexpectedly.

DST fold - America/New_York (1 November 2026)

At the autumn transition, 01:30 occurs twice. The converter detects more than one possible instant; it skips that local time instead of guessing which occurrence the guest intended.

Clock mismatch

The five-minute hold expiry is set with PostgreSQL clock_timestamp(). The UI countdown is only guidance and uses a server-provided time anchor where available; the authoritative acceptance decision is never based on the guest device clock.

User journeys and optional additions

Authenticated host

Sign in / out
Edit or disable event types
Multiple weekly windows / overrides
Bookings in host time zone
Cancel with reason and inspect outbox

Public guest

Browse host and event types
Select display time zone
Hold + countdown / release seat
Confirm and copy private link
Reschedule / cancel / retry

Navigation and accessibility

The public pages include Home, Browse Hosts, host event listings, private Manage Booking and Host Dashboard. Mobile navigation uses a full hamburger menu rather than silently hiding core links. Controls have labels and error feedback; a complete screen-reader audit is still outstanding.

Optional: abuse protection

Per-IP rate limits on public hold and host login routes reduce casual abuse. The implementation is intentionally single-process only; a distributed reverse-proxy or centralized store would be necessary for multi-instance production guarantees.

Optional: observability

Privacy-preserving request logs include method, route template, HTTP status and request ID without embedding private management tokens. Authenticated host counters expose confirmed, held, expired, cancelled and outbox event totals.

Core reliability worker

A periodic hold-cleanup worker releases abandoned holds. Availability reads independently ignore stale holds and do not impose host-row write locks. Writes still serialize at the host row and enforce native database constraints.

Test execution and reproducibility

The difference between authored tests and measured results is deliberate and visible.

Validation	Status / evidence
Prior Windows TypeScript build + startup	PASS (before release 2; observed from supplied logs)
Release 2 HTTP-contract tests	3 / 3 PASS in authoring container
Release 2 independent DST checks	4 / 4 PASS in authoring container
TS/TSX source syntax parsing	PASS, zero parser diagnostics
Release 2 npm build and React typecheck	NOT RUN here; npm registry/dependency restriction
Release 2 PostgreSQL integration tests	NOT RUN here; Windows runner required
Release 2 50-request HTTP proof	NOT RUN here; expected 1 x 201, 49 x 409
Release 2 real-browser/mobile accessibility	NOT RUN; manual acceptance checklist included

Manual Windows commands

After configuring the local .env for slotbook_ts / slotbook_ts_test: npm install, npm run build, npm run start:local. In another terminal: npm run test:local, followed by node --env-file=.env dist/scripts/concurrency.js --attempts 50.

Safety boundaries

The test fixture only accepts database names ending in _test before truncating test host records. Application migrations are forward-only and do not reset production data. The older Python result (28 passing tests, 1/49 concurrency) cannot be cited as validation of this TypeScript build.

AI use, known gaps and final acceptance

How AI was used

AI helped translate the brief into architecture, build the TypeScript React/Fastify prototype, design PostgreSQL locking/constraints, draft tests and report trade-offs. User-reported Windows errors prompted manual source corrections to error handling, frontend API state and a database safety guard. A later full-product audit led to the coordinated release 2 improvements.

What AI initially got wrong

An early artifact was described too confidently before its TypeScript build and real PostgreSQL runtime were verified. The original Python implementation did not satisfy the brief's TypeScript constraint. The first TypeScript build had compile errors, and the browser logout sent a JSON header with no body. These issues were corrected in source and disclosed; full fresh regression must still be measured.

Remaining gaps

No public repository URL or deployment URL is claimed. The source ZIP has no generated package-lock file because npm dependencies could not be installed in the authoring container. Real email, calendar synchronization, payments and password reset are outside the stated core. Public deployment requires HTTPS, Secure cookies, distributed abuse protection, accessibility review and production operational policies.

Release checklist

1) Fresh Windows npm install/build. 2) Apply and verify migration 002 on slotbook_ts. 3) Run all PostgreSQL tests on slotbook_ts_test. 4) Run the 50-request HTTP proof. 5) Complete host/guest desktop/mobile UAT. 6) Commit the generated lockfile and real evidence. 7) Publish unsquashed GitHub history and add its exact URL to this report.

Final principle

Correctness comes from PostgreSQL-backed invariants and atomically committed booking transitions. UI, retries and presentation should expose those guarantees without claiming test evidence that has not been observed.